

Pthreads

A low-level, OS-supported, kernel-managed threading interface that enables multiple threads of control to execute concurrently within the same process address space.

Comparison with Other Parallel Models Pthreads is Shared memory and for CPU parallel.

Process Memory Layout (Execution model)

Code (text)

Global variables

Heap

Thread 1 stack

Thread 2 stack

Thread 3 stack

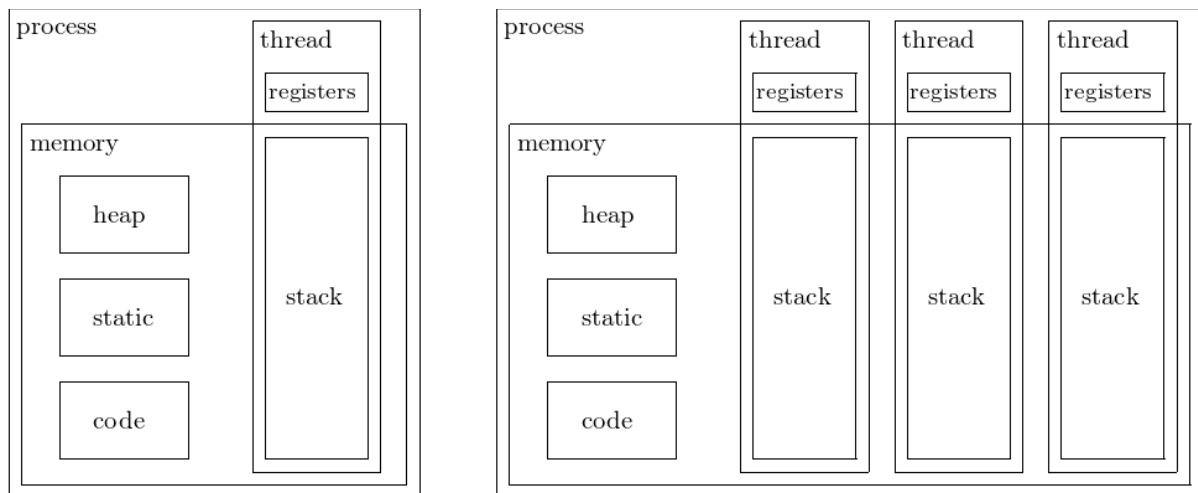


Fig 1: [Introduction to Pthreads](#)

- 1. Thread Creation, Management & Lifecycle**
- 2. Synchronization Overhead & Its Impact on Parallel Speed**
- 3. Load Balancing Techniques**
- 4. Parallel Patterns & Implementations Using pthreads**

Thread Creation, Management & Lifecycle (program example)

Pthreads provide fine-grained control over how threads are created, behave, and terminate.

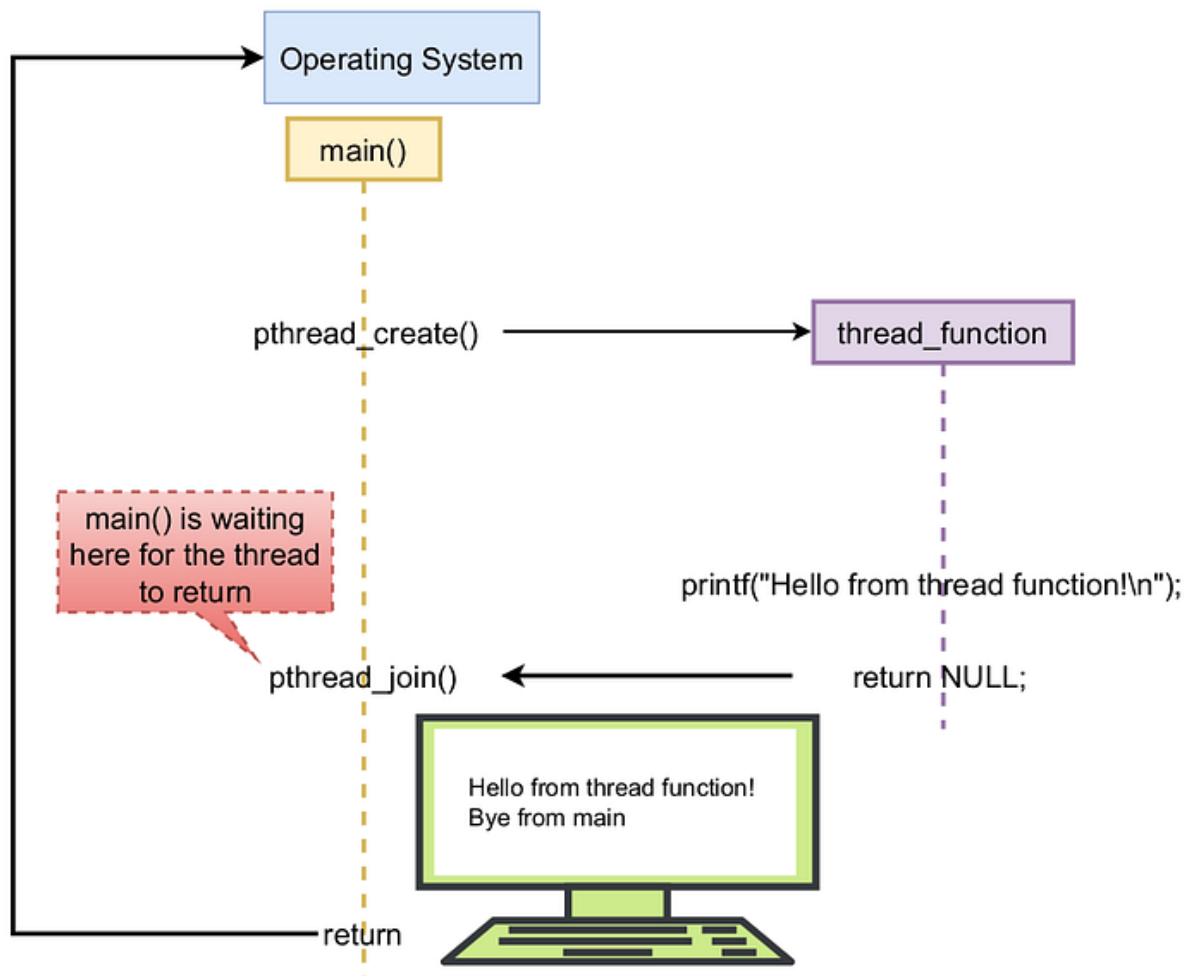


Fig 2: [pthread](#)

1. Thread Attributes (**pthread_attr_t**)

pthread_attr_t allows configuring a thread before creation. It does not affect existing threads—only new threads created with it.

a. Detach State (Detached vs Joinable)

Every thread starts as **joinable** unless explicitly detached.

Joinable threads: Must be waited on with **pthread_join()**, Allow capturing the return value, Consume system resources (stack + control structures) until joined.

Detached threads: Free their resources automatically when they finish, Cannot be joined, and return values are discarded, Often used for background tasks or fire-and-forget work.

```
pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);
```

b. Stack Size Tuning

Threads have their own stacks, typically 2–8 MB. For large local arrays or deep recursion, increasing stack size prevents overflow.

```
pthread_attr_setstacksize(&attr, size);
```

For thousands of threads, **reducing** stack size can save memory.

c. Scheduling Policies (SCHED_FIFO, SCHED_RR)

Pthreads allow setting real-time scheduling policies (requires privileges):

SCHED_FIFO: First-in-first-out, non-preemptive except by higher priority, Suitable for time-critical tasks

SCHED_RR: Round-robin among equal-priority threads, Ensures fairness while maintaining real-time behavior

```
pthread_attr_setschedpolicy(&attr, SCHED_FIFO);
```

d. CPU Affinity (Binding to Cores)

Linux offers `pthread_setaffinity_np()` to bind threads to specific cores.

Benefits: Improves cache locality, Reduces thread migration, Helps NUMA-aware performance

Example:

```
cpu_set_t set;
CPU_ZERO(&set);
CPU_SET(core_id, &set);
pthread_setaffinity_np(thread, sizeof(set), &set);
```

Affinity is crucial in high-performance and scientific applications.

2. Thread Cancellation

Thread cancellation lets one thread terminate another safely.

a. Deferred vs Asynchronous Cancellation

Deferred cancellation (default): Thread checks for cancellation at "cancellation points", Safe and predictable.

`pthread_join`, `sleep`, `read`, `printf`, etc.

Asynchronous cancellation: Thread is terminated immediately, Dangerous: can terminate while holding locks → deadlocks or corruption, Only used in exceptional situations.

```
pthread_setcanceltype(PTHREAD_CANCEL_DEFERRED, NULL);
pthread_setcanceltype(PTHREAD_CANCEL_ASYNCHRONOUS, NULL);
```

b. Cancellation Points

A thread only responds to cancellation at specific library functions when using deferred cancellation.

```
pthread_testcancel();
```

c. Cleanup Handlers (pthread_cleanup_push)

```
pthread_cleanup_push(cleanup_function, arg);  
  
// Critical region  
  
pthread_cleanup_pop(1); // Execute cleanup
```

Github program example: <https://github.com/HopingR/pthread.git>

Synchronization Overhead & Its Impact on Parallel Speed (conceptual based)

The extra time and performance cost introduced when multiple threads or processes coordinate with each other.

Ideal speed up

Threads	Ideal speed up
2	2*
4	4*
8	8*

Synchronization overhead makes real speedup look like:

Threads	Ideal speed up
2	1.7*
4	3.0*
8	5.2*

Because more threads → more coordination → more overhead.

Tree-based barrier (scales better)

Organize threads in a tree (e.g., binary tree). Arrival propagates up; release propagates down. Below is a sketch of a sense-reversing tree idea (simplified; production code needs careful memory ordering):

```
// Sketch only — not drop-in safe code.
```

```
// Idea: threads pair up; children signal parent when both arrived.
```

```
// Parent signals children to proceed on release phase.
```

```
*****
```

Mutex:

```
arrived[parent]++
```

```
if arrived[parent] == children_count: // Mutex protects only tiny, local updates
```

```
    signal parent
```

Ensures consistency of shared state at a node

A tree barrier spreads the synchronization work across many small nodes,

Only *two children* report to a parent node.

Eg: So instead of 32 threads contending on one lock, you get:

- level 1: pairs of 2 threads
- level 2: pairs of parents
- ...
- root handles only one update from each side

This reduces contention drastically.

Reduces contention on a single atomic variable, replacing $O(N)$ hotspot with $O(\log N)$ steps and distributed atomics.

Load Balancing Techniques (demerit-conceptual based)

Avoid idle threads during work distribution: Static scheduling, Dynamic scheduling, Work-stealing, Chunk size tuning.

When you use pthreads, you often create multiple threads to work in parallel. If one thread gets more work than others, it becomes the bottleneck.

Work Stealing

Each thread has its own deque(double-ended Queue) of tasks; it pushes/pops from the bottom (LIFO) locally (fast), while thieves steal from the top (FIFO) of other deques when they run out of work. This reduces contention vs. a single central queue and preserves locality.

Threads do their own work first → if they finish → they steal work from others. → creates automatic load balancing.

Thread 0: [task, task, task]

Thread 1: [task]

Thread 2: []

Thread 3: [task, task]

Thread 2 is idle → it steals a task from Thread 0

Every thread tries to stay busy by grabbing leftover work.

```
typedef struct {
```

```
    long *buf; // circular buffer of tasks (indices modulo cap)
```

```
    int cap; // capacity of buf
```

```
    int top; // steal end (FIFO head) -- thieves read/remove here
```

```
    int bot; // owner end (LIFO tail) -- owner push/pop here
```

```
    pthread_mutex_t m; // (only for the simple, lock-based version) to serialize access to the shared
```

deque state (top, bot, and buf) so that only one thread at a time can modify

```
} deq_t;
```

```
bool deq_push_bot(deq_t* d, long x);
```

```
bool deq_pop_bot(deq_t* d, long* x);    // owner only
```

```
bool deq_steal_top(deq_t* d, long* x);  // thieves
```

Parallel Patterns & Implementations Using pthreads(conceptual based)

Using pthreads to implement well-known parallel programming structures that appear again and again in high-performance computing (HPC). (**Parallel patterns** are *common reusable ways* to structure parallel program). **pthreads** gives you the raw tools (threads, locks, condition variables), and the patterns show you **how to assemble them into real parallel algorithms**.

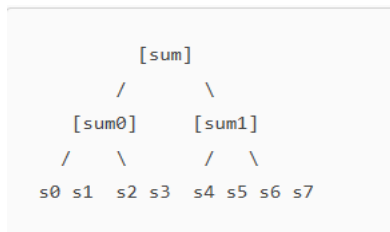
These are standard HPC parallel patterns:

Parallel for loop (OpenMP-style, but handcrafted) Producer-consumer pattern with bounded buffers
Task queues and worker threads Parallel prefix (scan) Parallel reduction Parallel sorting

Parallel Reduction

Reductions compute a single global value from many elements (sum, max, min).

Parallelization comes from **tree-like combination**.



thread computes local sum

barrier

thread 0 combines thread_sums[]

Two-Phase (Thread-Local + Combine): pseudocode

function parallel_reduce_sum(A[0..N-1], num_threads T):

shared partial_sums[0..T-1] // ideally padded to avoid false sharing

barrier B(T) //pthread_barrier_wait(&B) → (All participating threads wait until all arrive)

parallel_for thread_id t in [0..T-1]:

// compute chunk bounds (static partitioning)

chunk_start = floor(t * N / T)

chunk_end = floor((t+1) * N / T)

```

// local accumulation (register-resident)
local = 0
for i in [chunk_start .. chunk_end-1]: // phase 1
    local = local + A[i]
// write local to shared slot
partial_sums[t] = local
// wait for all threads
barrier_wait(B)
// one thread combines all partial sums
if t == 0:
    total = 0
    for k in [0 .. T-1]: //phase 2
        total = total + partial_sums[k]
    return total

```

Simple and efficient when you have T threads and a large array(N). Combines at the end in a single thread (or small tree).

Phase 1: Each thread reduces its chunk to a local partial (no locks).

Barrier

Phase 2: One thread combines the T partials.

Complexity: phase I local partial each thread does: N/T operations

$$Time\{local\} = O(N/T)$$

Phase 2: One thread (typically thread 0) performs the final combine, so its T iterations

$$Time\{combine\} = O(T)$$

$$Time\{total\} = O\left(\frac{N}{T}\right) + O(T)$$

Note: AI is used for conceptual explanation and program code example